

Essential NumPy Functions Guide

Sorting, Concatenation, Indexing, and Analysis

Data Science Reference

June 18, 2026

Contents

1	Introduction to NumPy Functions	3
2	Array Manipulation Functions	3
2.1	np.sort() - Sorting Arrays	3
2.1.1	Definition	3
2.1.2	Key Points	3
2.1.3	Examples	3
2.1.4	Sorting Algorithm Comparison	3
2.2	np.append() - Adding Elements	4
2.2.1	Definition	4
2.2.2	Key Points	4
2.2.3	Examples	4
2.3	np.concatenate() - Joining Multiple Arrays	4
2.3.1	Definition	4
2.3.2	Key Points	4
2.3.3	Examples	4
2.4	np.unique() - Finding Distinct Values	5
2.4.1	Definition	5
2.4.2	Key Points	5
2.4.3	Examples	5
3	Indexing and Selection Functions	6
3.1	np.where() - Conditional Selection	6
3.1.1	Definition	6
3.1.2	Examples	6
3.2	np.argmax() and np.argmin() - Finding Extrema	6
3.2.1	Definition	6
3.2.2	Examples	6
4	Statistical and Analysis Functions	7
4.1	np.cumsum() - Cumulative Sum	7
4.1.1	Definition	7
4.1.2	Examples	7
4.2	np.percentile() - Percentile Calculation	7
4.2.1	Definition	7
4.2.2	Examples	7
4.3	np.histogram() - Histogram Computation	8
4.3.1	Definition	8
4.3.2	Examples	8

5	Function Comparison Tables	8
5.1	Concatenation Functions Comparison	8
5.2	Indexing and Selection Functions	9
6	Quick Reference Cheat Sheet	9
7	Best Practices and Tips	9
7.1	Performance Tips	9
7.2	Common Mistakes to Avoid	9

1 Introduction to NumPy Functions

NumPy provides a rich set of functions for array manipulation, mathematical operations, and data analysis. This guide covers the most essential functions with clear definitions and practical examples.

2 Array Manipulation Functions

2.1 np.sort() - Sorting Arrays

2.1.1 Definition

`np.sort(a, axis=-1, kind=None, order=None)` returns a sorted copy of an array along a specified axis, arranging elements in ascending order while leaving the original array unchanged.

2.1.2 Key Points

- Returns a new array – original remains unmodified
- Default: sorts along the last axis
- Ascending order only
- Efficient: uses $O(N \log N)$ algorithms like quicksort

2.1.3 Examples

Listing 1: np.sort Examples

```
1 import numpy as np
2
3 # 1D Array
4 arr = np.array([5, 2, 8, 1, 9, 3])
5 sorted_arr = np.sort(arr)
6 print("Original:", arr)          # [5 2 8 1 9 3]
7 print("Sorted:", sorted_arr)     # [1 2 3 5 8 9]
8
9 # 2D Array (row-wise)
10 arr_2d = np.array([[3, 1, 4], [6, 2, 5]])
11 row_sorted = np.sort(arr_2d, axis=1)
12 print("Row-wise sorted:\n", row_sorted)
13 # [[1 3 4]
14 #   [2 5 6]]
15
16 # Sorting vs in-place
17 arr = np.array([3, 1, 2])
18 np.sort(arr)                    # Returns new array
19 arr.sort()                      # Modifies original array
```

2.1.4 Sorting Algorithm Comparison

Algorithm	Speed	Worst Case	Stable
'quicksort' (default)	Fastest	$O(n^2)$	No
'mergesort'	Good	$O(n \log n)$	Yes
'heapsort'	Slower	$O(n \log n)$	No
'stable'	Good	$O(n \log n)$	Yes

Table 1: Sorting Algorithm Options

2.2 np.append() - Adding Elements

2.2.1 Definition

`np.append(arr, values, axis=None)` appends values to the end of an array along a specified axis, returning a new array without modifying the original.

2.2.2 Key Points

- Returns a new array – original remains unchanged
- Data types must be compatible
- Inefficient in loops – recreates entire array each time
- Without `axis`, flattens everything to 1D

2.2.3 Examples

Listing 2: np.append Examples

```
1 # 1D Appending
2 arr = np.array([1, 2, 3])
3 new_arr = np.append(arr, 4)
4 print("Original:", arr)           # [1 2 3]
5 print("New array:", new_arr)      # [1 2 3 4]
6
7 # 2D Appending (add row)
8 arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
9 new_row = np.array([[7, 8, 9]])
10 row_appended = np.append(arr_2d, new_row, axis=0)
11 print("New row added:\n", row_appended)
12 # [[1 2 3]
13 #  [4 5 6]
14 #  [7 8 9]]
15
16 # 2D Appending (add column)
17 new_col = np.array([[10], [11]])
18 col_appended = np.append(arr_2d, new_col, axis=1)
19 # [[1 2 3 10]
20 #  [4 5 6 11]]
```

2.3 np.concatenate() - Joining Multiple Arrays

2.3.1 Definition

`np.concatenate((a1, a2, ...), axis=0)` joins a sequence of arrays along an existing axis, combining multiple arrays into a single array without flattening.

2.3.2 Key Points

- Joins multiple arrays at once
- All arrays must have same shape except along concatenation axis
- Maintains dimensions (no flattening)
- Faster than repeated `np.append()`

2.3.3 Examples

Listing 3: np.concatenate Examples

```

1 # 1D Concatenation
2 arr1 = np.array([1, 2, 3])
3 arr2 = np.array([4, 5, 6])
4 arr3 = np.array([7, 8, 9])
5 result = np.concatenate([arr1, arr2, arr3])
6 print(result) # [1 2 3 4 5 6 7 8 9]
7
8 # 2D Vertical (axis=0)
9 arr1 = np.array([[1, 2, 3], [4, 5, 6]])
10 arr2 = np.array([[7, 8, 9], [10, 11, 12]])
11 vertical = np.concatenate([arr1, arr2], axis=0)
12
13 # 2D Horizontal (axis=1)
14 horizontal = np.concatenate([arr1, arr2], axis=1)
15
16 # Convenience functions
17 vstack_result = np.vstack([arr1, arr2]) # Same as axis=0
18 hstack_result = np.hstack([arr1, arr2]) # Same as axis=1

```

2.4 np.unique() - Finding Distinct Values

2.4.1 Definition

`np.unique(ar, return_index=False, return_inverse=False, return_counts=False, axis=None)` returns the sorted unique values from an array, identifying all distinct elements and removing duplicates.

2.4.2 Key Points

- Returns sorted unique values
- Can count occurrences of each value
- Can reconstruct original array using inverse indices
- Works on multi-dimensional arrays

2.4.3 Examples

Listing 4: np.unique Examples

```

1 arr = np.array([1, 3, 2, 3, 1, 2, 4, 2, 1, 5])
2
3 # Basic unique
4 unique_vals = np.unique(arr) # [1 2 3 4 5]
5
6 # With counts
7 values, counts = np.unique(arr, return_counts=True)
8 print("Values:", values) # [1 2 3 4 5]
9 print("Counts:", counts) # [3 3 2 1 1]
10
11 # With inverse (reconstruct)
12 values, inverse = np.unique(arr, return_inverse=True)
13 reconstructed = values[inverse] # Reconstructs original
14
15 # 2D array - unique rows
16 arr_2d = np.array([[1, 2, 1], [2, 3, 2], [1, 2, 1]])
17 unique_rows = np.unique(arr_2d, axis=0)
18 # [[1 2 1]
19 # [2 3 2]]
20
21 # Practical: Label encoding for machine learning
22 categories = np.array(['red', 'blue', 'red', 'green'])
23 colors, labels = np.unique(categories, return_inverse=True)

```

```

24 # colors: ['blue', 'green', 'red']
25 # labels: [2, 0, 2, 1]

```

3 Indexing and Selection Functions

3.1 np.where() - Conditional Selection

3.1.1 Definition

`np.where(condition, [x, y])` returns elements chosen from `x` or `y` depending on a condition, or returns indices where the condition is `True`.

3.1.2 Examples

Listing 5: np.where Examples

```

1 arr = np.array([1, 2, 3, 4, 5, 6])
2
3 # Get indices where condition is true
4 indices = np.where(arr > 3)
5 print(indices) # (array([3, 4, 5]),)
6
7 # Replace based on condition
8 result = np.where(arr > 3, arr * 2, arr)
9 print(result) # [1, 2, 3, 8, 10, 12]
10
11 # Replace values (clean data)
12 temps = np.array([22, 25, -999, 28, -999, 24])
13 cleaned = np.where(temps == -999, np.nan, temps)
14 print(cleaned) # [22, 25, nan, 28, nan, 24]
15
16 # Multiple conditions
17 arr_2d = np.array([[1, 2], [3, 4]])
18 result = np.where(arr_2d > 2, 'High', 'Low')
19 # [['Low', 'Low'], ['High', 'High']]

```

3.2 np.argmax() and np.argmin() - Finding Extrema

3.2.1 Definition

`np.argmax(a, axis=None)` returns the indices of the maximum values along an axis, while `np.argmin()` returns indices of minimum values.

3.2.2 Examples

Listing 6: np.argmax and np.argmin Examples

```

1 arr = np.array([10, 25, 5, 30, 15])
2
3 print(np.argmax(arr)) # 3 (index of max value 30)
4 print(np.argmin(arr)) # 2 (index of min value 5)
5
6 # 2D Array
7 arr_2d = np.array([[8, 3, 5], [1, 9, 4], [2, 7, 6]])
8
9 # Max in each row
10 row_max_indices = np.argmax(arr_2d, axis=1) # [0, 1, 1]
11 row_max_values = arr_2d[np.arange(3), row_max_indices] # [8, 9, 7]
12
13 # Max in each column
14 col_max_indices = np.argmax(arr_2d, axis=0) # [0, 1, 2]
15 col_max_values = arr_2d[col_max_indices, np.arange(3)] # [8, 9, 6]

```

```

16
17 # Finding overall max
18 overall_index = np.argmax(arr_2d) # 4 (flattened index)
19 overall_max = arr_2d.flat[overall_index] # 9

```

4 Statistical and Analysis Functions

4.1 np.cumsum() - Cumulative Sum

4.1.1 Definition

`np.cumsum(a, axis=None)` returns the cumulative sum of elements along a specified axis, where each element is the sum of all previous elements up to that point.

4.1.2 Examples

Listing 7: np.cumsum Examples

```

1 arr = np.array([1, 2, 3, 4, 5])
2 cumsum = np.cumsum(arr)
3 print(cumsum) # [1, 3, 6, 10, 15]
4
5 # 2D Array
6 arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
7
8 # Cumulative sum along rows
9 row_cumsum = np.cumsum(arr_2d, axis=1)
10 # [[1, 3, 6],
11 #   [4, 9, 15]]
12
13 # Cumulative sum along columns
14 col_cumsum = np.cumsum(arr_2d, axis=0)
15 # [[1, 2, 3],
16 #   [5, 7, 9]]
17
18 # Cumulative sum (flattened)
19 flat_cumsum = np.cumsum(arr_2d) # [1, 3, 6, 10, 15, 21]

```

4.2 np.percentile() - Percentile Calculation

4.2.1 Definition

`np.percentile(a, q, axis=None)` computes the q-th percentile of the data along a specified axis, where q is a value between 0 and 100.

4.2.2 Examples

Listing 8: np.percentile Examples

```

1 data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
2
3 # Single percentile
4 median = np.percentile(data, 50) # 5.5
5 q25 = np.percentile(data, 25) # 3.25
6 q75 = np.percentile(data, 75) # 7.75
7
8 # Multiple percentiles
9 percentiles = np.percentile(data, [25, 50, 75])
10 print(percentiles) # [3.25, 5.5, 7.75]
11
12 # 2D Array
13 arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

```

```

14 col_percentiles = np.percentile(arr_2d, 50, axis=0) # [4, 5, 6]
15 row_percentiles = np.percentile(arr_2d, 50, axis=1) # [2, 5, 8]
16
17 # Interquartile Range (IQR)
18 q1 = np.percentile(data, 25)
19 q3 = np.percentile(data, 75)
20 iqr = q3 - q1
21 print(f"IQR: {iqr}") # 4.5

```

4.3 np.histogram() - Histogram Computation

4.3.1 Definition

`np.histogram(a, bins=10, range=None)` computes the histogram of a dataset, returning the histogram values and bin edges.

4.3.2 Examples

Listing 9: np.histogram Examples

```

1 data = np.array([1, 2, 2, 3, 3, 3, 4, 4, 5])
2
3 # Basic histogram
4 hist, bin_edges = np.histogram(data, bins=5)
5 print("Histogram counts:", hist) # [1, 2, 3, 2, 1]
6 print("Bin edges:", bin_edges) # [1, 1.8, 2.6, 3.4, 4.2, 5]
7
8 # Specifying bin edges manually
9 bins = [1, 2, 3, 4, 5, 6]
10 hist, bin_edges = np.histogram(data, bins=bins)
11 print("With custom bins:", hist) # [1, 2, 3, 2, 1]
12
13 # With range
14 hist, bin_edges = np.histogram(data, bins=5, range=(1, 5))
15
16 # For continuous data
17 continuous = np.random.randn(1000)
18 hist, bin_edges = np.histogram(continuous, bins=20)
19
20 # Using with matplotlib
21 import matplotlib.pyplot as plt
22 plt.hist(data, bins=5, alpha=0.7)
23 plt.title('Histogram of Data')
24 plt.show()

```

5 Function Comparison Tables

5.1 Concatenation Functions Comparison

Function	What it does	Equivalent to
<code>np.concatenate()</code>	Join arrays along existing axes	<code>np.concatenate([...], axis=0)</code>
<code>np.vstack()</code>	Stack vertically	<code>np.concatenate(..., axis=0)</code>
<code>np.hstack()</code>	Stack horizontally	<code>np.concatenate(..., axis=1)</code>
<code>np.dstack()</code>	Stack depth-wise	<code>np.concatenate(..., axis=2)</code>
<code>np.append()</code>	Append values to array	Simplified concatenation

Table 2: Concatenation Functions

5.2 Indexing and Selection Functions

Function	Returns	Common Use
<code>np.where()</code>	Indices or conditional values	Conditional selection
<code>np.argmax()</code>	Index of maximum	Finding best performer
<code>np.argmin()</code>	Index of minimum	Finding worst performer
<code>np.unique()</code>	Unique values with options	Data cleaning

Table 3: Indexing and Selection Functions

6 Quick Reference Cheat Sheet

Table 4: Essential NumPy Functions Quick Reference

Function	What it does	Code Example
<code>np.sort()</code>	Sorted copy	<code>np.sort([3,1,2])</code> → <code>[1,2,3]</code>
<code>np.append()</code>	Add elements	<code>np.append([1,2], 3)</code> → <code>[1,2,3]</code>
<code>np.concatenate()</code> <code>np.unique()</code>	Join multiple Distinct values	<code>np.concatenate([a,b,c])</code> <code>np.unique([1,1,2])</code> → <code>[1,2]</code>
<code>np.where()</code>	Conditional	<code>np.where(arr<5, arr, 0)</code>
<code>np.argmax()</code>	Max index	<code>np.argmax([1,3,2])</code> → <code>1</code>
<code>np.argmin()</code>	Min index	<code>np.argmin([1,3,2])</code> → <code>0</code>
<code>np.cumsum()</code>	Cumulative sum	<code>np.cumsum([1,2,3])</code> → <code>[1,3,6]</code>
<code>np.percentile()</code>	Percentile	<code>np.percentile(data, 50)</code>
<code>np.histogram()</code>	Frequency counts	<code>np.histogram(data, bins=5)</code>

7 Best Practices and Tips

7.1 Performance Tips

- Use `np.concatenate()` instead of repeated `np.append()` in loops
- Use `np.unique()` with `return_counts=True` for frequency analysis
- Vectorized operations (like `np.where()`) are faster than loops
- For large arrays, `np.unique()` is more efficient than `set()`
- Use `arr.sort()` for in-place sorting when you don't need the original

7.2 Common Mistakes to Avoid

- Forgetting that `np.append()` returns a new array (doesn't modify original)

- Shape mismatch in `np.concatenate()` — all arrays must have same dimensions
- Using `np.append()` in loops causing $O(n^2)$ complexity
- Using `np.where()` without specifying x and y when you want indices
- Not handling NaN values before using `np.percentile()` or `np.histogram()`